

Spring Boot Coding Standard — v2.0

Owner: Naveen · **Approved:** Praveen (CTO)
Stack: Java 21 · Spring Boot 3.x · PostgreSQL · Maven
Supersedes: Spring Boot Coding Standard v1.0

Scope and status

This standard applies to **new and changed code**. Existing code is not retrofitted — a rule arriving here does not create a backlog of rewrites. When you touch a file, the code you write in it follows this document.

Sections 1–12 mirror the Angular, Next.js, Node and Python standards section for section. Sections 13 and 14 are backend-specific.

Version policy

Projects run a supported LTS Java and a supported Spring Boot minor. Upgrades are planned work, scheduled at least once per release cycle, not deferred until something forces them. Spring Boot minors carry security fixes and go out of support faster than teams expect.

Enforcement tags

Tag	Meaning
[LINTER]	A tool in the repo catches it — Checkstyle, SpotBugs, ArchUnit, the compiler.
[CI]	A pipeline step or Git-host setting catches it — Gitleaks, OWASP dependency-check, JaCoCo, branch protection.
[NEITHER]	A human must judge it in review.
[LINTER + NEITHER]	The tool covers part; review covers the rest.
[CI + NEITHER]	The pipeline covers part; review covers the rest.

ArchUnit is the highest-leverage tool in this stack. Most of the layering and dependency rules below are expressible as ArchUnit tests, which run as part of the normal test suite — see the appendix.

1. Naming

- [LINTER] **Packages grouped by feature, not by layer:** `com.company.<project>.property.*`, never `...controller.*`. *Reason:* when a feature is deleted or handed over, everything about it is in one folder. **Check:** ArchUnit package rule.
- [NEITHER] A `PropertyService` **interface** only when a second implementation actually exists. One implementation behind an interface is indirection nobody asked for.
- [NEITHER] **DTOs name their direction:** `CreatePropertyRequest`, `PropertyResponse`, `PropertySummaryView`. Never `PropertyDTO` — it tells the reader nothing.
- [NEITHER] **Methods returning `Optional` start with `find`; methods that throw when missing start with `get`.** *Reason:* the caller knows the failure mode from the call site, without opening the implementation. *(This convention is used across every stack in the company.)*
- [LINTER] Booleans read as questions — `isActive`, `hasExpired`. DB columns `snake_case`, fields `camelCase`, mapping always explicit via `@Column`. **Check:** Checkstyle naming rules.
- [LINTER] No magic numbers or repeated string literals in service code — a named constant or config. **Check:** Checkstyle `MagicNumber`, SonarQube duplicate-literal rule.

1.1 Wire format across stacks

[NEITHER] **JSON payloads are camelCase on the wire, in both directions.**

Java is already camelCase, so nothing converts here — but the rule is stated so it is not quietly broken by a `@JsonProperty("property_name")` added to match one consumer. Our Python services convert at their boundary; Java does not.

Database columns stay `snake_case`, mapped explicitly with `@Column`. The database convention and the wire convention are separate decisions and both are deliberate.

2. Project structure

```
src/main/java/com/company/<project>/
├── property/
│   ├── route/      PropertyRoutes.java      (path constants: /api/v1/properties)
│   ├── controller/ PropertyController.java
│   ├── service/    PropertyService.java
│   ├── repository/ PropertyRepository.java
│   ├── dto/        CreatePropertyRequest.java · PropertyResponse.java
│   └── entity/      Property.java · PropertyStatus.java (enum)
├── user/ · booking/ · payment/              (every feature has the same six folders)
├── common/
│   ├── exception/  GlobalExceptionHandler · ApiError · BusinessException
│   ├── config/     SecurityConfig · JacksonConfig · OpenApiConfig
│   ├── observability/ CorrelationIdFilter · AlertService
│   └── money/       Money.java
└── Application.java

src/main/resources/    application.yml + -dev/-staging/-prod · db/migration/V1__init.sql
src/test/java/...      mirrors the main tree exactly
```

[LINTER] Routes → Controller → Service → Repository, one direction only.

- Every URL path lives as a constant in the feature's `route/` class and the controller references it — never a hard-coded string on `@RequestMapping`. A route path appears exactly once in the codebase, so renaming an endpoint is one edit and every path is greppable.
- A controller must never inject a repository.
- An entity must never leave the service layer. Returning an entity means Hibernate lazy-loads during JSON serialisation and every column silently becomes public API.
- Entity-to-DTO conversion is a static factory on the DTO — `PropertyResponse.from(property)`. No mapper class, and not inline in the service.

Check: ArchUnit — all four are enforceable and are in the appendix.

3. Error handling & validation

- [LINTER] Shape validation at the edge with `@Valid` on the request DTO; business rules in the service. Two kinds of invalid deserve two different status codes — 400 for malformed, 409 for a well-formed request that conflicts with current state.
- [LINTER] One `@RestControllerAdvice` for the whole app. No `try/catch` inside controllers. Check: ArchUnit.
- [NEITHER] Every error response has the same body: `timestamp, status, code, message, path, correlationId, fieldErrors[]`. The frontends write one parser, not one per endpoint.
- [LINTER] Never `catch (Exception e) and continue`. If you cannot handle it, do not catch it. Never `e.printStackTrace()`. Check: SpotBugs, Checkstyle `IllegalCatch`.
- [NEITHER] Never return `null` to mean “not found” — throw `ResourceNotFoundException`. Never rethrow as `new RuntimeException(e.getMessage())`; that throws away the cause and the type.
- [NEITHER] Stack traces and SQL messages never reach the client. Log them with the correlation id, **return the id**.
- [NEITHER] Every public route is versioned (`/api/v1/...`). Inside a version you may add optional fields; removing or renaming a field, or tightening validation, is a breaking change and needs a new version plus a note in the PR.

```
// common/exception/ApiError.java
public record ApiError(
    Instant timestamp,
    int status,
    String code,
    String message,
    // machine-readable: LOAN_ALREADY_SETTLED
    // developer-facing; never shown to a user
)
```

```
String path,
String correlationId,
Map<String, List<String>> fieldErrors
) {}
```

```
@RestControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(BusinessException.class)
    ResponseEntity<ApiError> handleBusiness(BusinessException ex, HttpServletRequest req) {
        if (ex.getStatus().is5xxServerError()) log.error("Business failure: {}", ex.getCode(), ex);
        else log.warn("Business rule rejected: {} at {}", ex.getCode(), req.getRequestURI());
        return ResponseEntity.status(ex.getStatus()).body(toApiError(ex, req));
    }

    @ExceptionHandler(MethodArgumentNotValidException.class)
    ResponseEntity<ApiError> handleValidation(MethodArgumentNotValidException ex, HttpServletRequest req) {
        var fieldErrors = ex.getBindingResult().getFieldErrors().stream()
            .collect(groupingBy(FieldError::getField,
                mapping(FieldError::getDefaultMessage, toList())));
        return ResponseEntity.badRequest()
            .body(error(400, "VALIDATION_FAILED", "Invalid request", req, fieldErrors));
    }

    @ExceptionHandler(Exception.class)
    ResponseEntity<ApiError> handleUnexpected(Exception ex, HttpServletRequest req) {
        // Anything unrecognised is ours, and someone needs to look at it.
        log.error("Unhandled exception at {}", req.getRequestURI(), ex);
        alerts.fire(AlertEvent.UNHANDLED_EXCEPTION, Map.of("path", req.getRequestURI()));
        return ResponseEntity.status(500)
            .body(error(500, "INTERNAL_ERROR", "An unexpected error occurred", req, Map.of()));
        // ^ never the real message
    }
}
```

Return the correlation id on every error. It costs nothing and it is what turns a support ticket into a log lookup. The frontends display it on their error screens.

[NEITHER] Never HTTP 200 with `{"success": false}`. Status codes exist; use them.

4. Logging, observability & alerting

4.1 Log levels

Level	Meaning
ERROR	Someone is woken up.
WARN	Recovered but suspicious.
INFO	Business events only — created, state changed, payment captured.
DEBUG	Local.

One INFO line per DB call means nobody reads INFO at all.

- [LINTER] SLF4J with parameters: `log.info("Property {} approved by {}", id, userId)`. String concatenation builds the message even when the level is off. Check: SpotBugs `SLF4J_FORMAT_SHOULD_BE_CONST`, Checkstyle regex on `log.*\+`.

4.2 Correlation IDs

[NEITHER] Every request gets a correlation id in the MDC and back in a response header. *Reason:* without it you cannot follow one user's failure through the logs.

```
// common/observability/CorrelationIdFilter.java
@Component
@Order(Ordered.HIGHEST_PRECEDENCE)
public class CorrelationIdFilter extends OncePerRequestFilter {

    public static final String HEADER = "X-Correlation-Id";
    public static final String MDC_KEY = "correlationId";

    @Override
    protected void doFilterInternal(HttpServletRequest req, HttpServletResponse res, FilterChain chain)
        throws ServletException, IOException {
        var id = Optional.ofNullable(req.getHeader(HEADER))
            .filter(s -> !s.isBlank())
            .map(s -> s.substring(0, Math.min(s.length(), 64)))
            .orElseGet(() -> UUID.randomUUID().toString());

        MDC.put(MDC_KEY, id);
        res.setHeader(HEADER, id);
        try {
            chain.doFilter(req, res);
        } finally {
            MDC.remove(MDC_KEY); // or it leaks into the next request on this thread
        }
    }
}
```

[NEITHER] **Outbound calls to our own services forward the header.** A chain that drops it at the second hop is no more useful than having none. Configure it once on the `RestClient` / `WebClient` bean, not per call.

4.3 Redaction enforced in code

[NEITHER] **Never logged:** passwords, JWTs, API keys, OTPs, card or bank data, full request bodies containing PII, `Authorization` headers.

A denylist in a document is a hope. Logback applies it on every event:

```
<!-- logback-spring.xml -->
<configuration>
  <appender name="JSON" class="ch.qos.logback.core.ConsoleAppender">
    <encoder class="net.logstash.logback.encoder.LogstashEncoder">
      <includeMdcKeyName>correlationId</includeMdcKeyName>
      <jsonGeneratorDecorator class="net.logstash.logback.decorate.MaskingJsonGeneratorDecorator">
        <path>password</path>      <path>passwd</path>
        <path>token</path>         <path>accessToken</path>
        <path>refreshToken</path> <path>authorization</path>
        <path>apiKey</path>        <path>secret</path>
        <path>otp</path>           <path>pin</path>
        <path>cvv</path>           <path>cardNumber</path>
        <path>aadhaar</path>       <path>pan</path>
        <path>ssn</path>           <path>email</path>
      </jsonGeneratorDecorator>
    </encoder>
  </appender>
  <root level="INFO"><appender-ref ref="JSON"/></root>
</configuration>
```

If something sensitive does reach a log, treat it as a leaked credential and rotate it. The log has already been shipped and retained. Removing the log line changes nothing.

- [NEITHER] **Production minimum:** JSON-structured logs to stdout, `/actuator/health` and `/actuator/metrics` on an internal path only. `/actuator/env`, `/heapdump` and Swagger UI are **never publicly reachable**.

4.4 Alerting — ERROR means someone is woken up

[NEITHER] **Every service has an error alerting system.** The level semantics above say ERROR wakes someone; this is the mechanism that does the waking. A log line nobody is watching is not an alert.

Minimum events that must raise an alert:

Event	Why
Unhandled exception	The service is degraded right now
Failed @Scheduled or batch job	Silent, and compounds daily
Failed third-party API or webhook processing	Our data is now out of sync with theirs
Connection pool exhaustion or database connection loss	Everything downstream is about to fail
Authentication or payment failures above a threshold	Either an outage or an attack

Alerts carry enough context to act on: environment, service, correlation id, error code and a short message. *An alert that only says "Error" costs more time than it saves.*

```
@Service
public class AlertService {

    private static final Duration WINDOW = Duration.ofMinutes(5);
    private final Map<String, Instant> seen = new ConcurrentHashMap<>();

    public void fire(AlertEvent event, Map<String, Object> context) {
        var payload = new LinkedHashMap<String, Object>(context);
        payload.put("event", event);
        payload.put("environment", props.environment()); // channels split by environment
        payload.put("service", props.serviceName());
        payload.put("correlationId", MDC.get(CorrelationIdFilter.MDC_KEY));

        log.error("ALERT {} {}", event, payload);

        // Rate-limited so a repeating failure does not flood the channel and get muted.
        var key = event + ":" + context.getDefault("code", "");
        var last = seen.get(key);
        if (last != null && Duration.between(last, Instant.now()).compareTo(WINDOW) < 0) return;
        seen.put(key, Instant.now());

        notifier.send(payload); // Teams webhook for this environment
    }
}
```

- [NEITHER] Alert channels are separated by environment. Production noise mixed with staging noise gets muted together.

5. Testing expectations

- [NEITHER] Any service method with a branch in it needs unit tests covering the failure paths, not just the happy one. That is where our production bugs come from.
- [NEITHER] Unit tests are required for: money, amount, interest and fee calculations; date and schedule logic; state-transition rules; shared utilities. These produce a wrong answer silently instead of failing, so no manual tester will catch them.
- [NEITHER] Every endpoint: at least one @WebMvcTest for the success case and its main error case, plus its auth-failure case.
- [CI] Custom repository queries: @DataJpaTest against Testcontainers PostgreSQL, not H2. H2 accepts queries Postgres rejects, so a green H2 test proves nothing.
- [CI] Coverage gate 70% on service and domain packages. DTOs and config excluded — coverage on getters is a vanity number. **Check:** JaCoCo rule in the Maven build.
- [NEITHER] Every bug fix ships with a test that fails without the fix. This is how we get a regression suite without ever scheduling a project to build one.
- [LINTER] No Thread.sleep(), and no test that depends on another test running first. **Check:** Checkstyle regex; Awaitility is the alternative.
- [CI] Tests never run against live third-party APIs or production data.
- [CI] Existing tests pass before merge. A developer does not disable or delete a failing test to get a PR through.

6. PR & merge rules

- [CI] One approval minimum, from someone other than the author, on every PR in every project. Two when the PR touches authentication,

payments, migrations, or `common/`. **Check:** branch protection plus CODEOWNERS.

- **[NEITHER]** **Around 400 changed lines maximum**, excluding generated and lock files. Past that, reviewers approve instead of read. Split it.
- **[CI]** Blocks merge: failing required CI, self-approval, protected-branch bypass, unresolved blocking comments, or **a migration with no rollback plan**.
- **[LINTER]** Squash merge. Branch name `feature/PROJ-123-short-slug`.
- **[CI + NEITHER]** **Required PR description fields:** what changed · why it changed · how it was tested · security/architecture impact · breaking changes · related ticket · deployment requirements. **A PR with an empty description does not get reviewed.** *“Tested locally” is not an answer for how it was tested.*
- **[CI]** Gitleaks and an OWASP dependency CVE scan run on every PR. **High severity blocks merge; Medium requires triage.** A detected secret blocks the merge **and triggers rotation** — once committed it is in the history.

On the review requirement for a sole-developer service. This stack currently has one developer. The rule is not waived — the named reviewer is the Node or Python lead, who reviews for logic, security and the rules in this document rather than for Spring idiom. A service nobody else has ever read is a continuity risk as much as a quality one.

7. Review checklist

1. **[NEITHER]** **Is the endpoint authorised, not just authenticated — and does it check ownership of the record?** (8.3)
2. **[NEITHER]** Request DTO validated, and is the response a DTO rather than an entity?
3. **[NEITHER]** Any N+1? **Check the query count in the test log, not by eye.**
4. **[NEITHER]** Is `@Transactional` on the service method, with `readOnly = true` on reads?
5. **[NEITHER]** Are the failure paths tested, or only the happy path?
6. **[NEITHER]** Do errors go through the advice with the correct HTTP status?
7. **[NEITHER]** **Money handled per section 12 — no double or float anywhere near an amount?**
8. **[LINTER + NEITHER]** Any secret, token or PII in code, logs, or test fixtures?
9. **[NEITHER]** New dependency — is it justified in the description?
10. **[NEITHER]** Is the migration safe to run on a populated table, and reversible?
11. **[NEITHER]** Anything hard-coded that belongs in config?
12. **[NEITHER]** Does each method do only what its name claims?
13. **[NEITHER]** Does this change need a new alert, and is it wired? (4.4)
14. **[NEITHER]** **Could the newest developer on the team read this without asking the author?**

8. Security & secrets

8.1 Secrets

- **[CI]** **No secrets in the repository, ever.** `.env` and `application-prod.yml` gitignored; `.env.example` committed with every key present and every value empty. **Check:** Gitleaks, blocking.
- **[NEITHER]** **Secrets come from environment variables (`#{DB_PASSWORD}`) with no default fallback** — defaults get shipped and then get forgotten.
- **[NEITHER]** **Any credential that ever reached a commit is rotated, even after a force-push.** The history is already on someone's laptop.

AWS: production secrets come from Secrets Manager or Parameter Store, bound through Spring Cloud AWS or injected as environment variables at container start.

8.2 Authentication

- **[NEITHER]** **BCrypt for passwords.** Short-lived JWT with refresh; signing key from the environment, minimum 256-bit.
- **[NEITHER]** **Everything issued for verification expires:** sessions, access and refresh tokens, OTPs, password-reset links, signed URLs. Logout, password change or role change invalidates existing sessions.

- [NEITHER] Refresh tokens rotate on use and the old token is invalidated. A reused refresh token is treated as replay: revoke the session and alert.
- [NEITHER] **CORS is an explicit origin list per environment** — never `*` alongside credentials. **CSRF is disabled only because we are stateless and token-based; that reason is written in `SecurityConfig`, not assumed.**

8.3 Authorisation — the ownership check

[NEITHER] `@PreAuthorize` plus an explicit ownership check in the service.

IDOR is our most likely real vulnerability: a valid token for user A must not be able to fetch user B's record by changing the id in the URL. It needs no tools — a user changes a number in the address bar and finds it by accident. Scanners miss it, because the request is well-formed, authenticated, and returns 200.

The fix is in how you query, not in a check you remember to add:

```
// ✗ returns anyone's property to anyone who is signed in
var property = repository.findById(id).orElseThrow(...);

// ✓ the query cannot return another user's record
var property = repository.findByIdAndOwnerId(id, currentUser.getId())
    .orElseThrow(() -> new ResourceNotFoundException("PROPERTY_NOT_FOUND", id));
```

Scope the query to the caller and there is no check to forget.

- [NEITHER] **Return 404, not 403, for a record the caller does not own.** A 403 confirms the record exists, which is itself a disclosure.
- [NEITHER] A role check is additional, not a substitute. An admin role still has an explicit check.
- [NEITHER] **Every PR that adds an endpoint names, in its description, how ownership is enforced on it.**

8.4 Request safety

- [LINTER] **Parameter binding only.** Never build JPQL or native SQL by string concatenation. **Check:** SpotBugs `SQL_INJECTION`.
- [NEITHER] Every inbound webhook verifies its signature or HMAC **before the payload is processed**. The URL is not a credential.
- [NEITHER] Rate limiting on auth, OTP, and any endpoint that costs money.
- [NEITHER] Request body size limits and security headers enabled in production.
- [NEITHER] File uploads restricted by size and by **actual content type**, not the filename or declared MIME type. Stored privately, served through expiring signed URLs.

8.5 Never trust the client

[NEITHER] Client-supplied identity, amount, price, role or status is re-derived or re-verified server-side. **Always.** A field the server does not verify is a field the server has delegated to an attacker.

9. Dependencies & configuration

- [NEITHER] **The Spring Boot BOM decides versions.** Do not pin an individual Spring artifact by hand — that is how you get a mismatched transitive set.
- [NEITHER] A new dependency needs, in the PR description: what it does, why the JDK or Spring cannot, its last release date, and its licence. **No library added for one utility method.**
- [NEITHER] **Nothing unmaintained** — no release in 18 months means it is our problem now, not theirs.
- [CI] **Maven wrapper committed.** A clean clone must build with `./mvnw verify` and nothing else installed.
- [CI] Dependency CVE scan on every PR; High severity blocks, Medium requires triage. Existing findings are baselined so CI blocks on new problems, not old ones. **The baseline is not regenerated to make a failing pipeline pass.**
- [LINTER] **Schema changes only through Flyway, forward-only.** An applied migration is never edited. `ddl-auto: validate` everywhere above `local` — `update` mutates the production schema silently and cannot be reviewed.
- [NEITHER] **Behaviour differences live in profiles**, not in `if (env.equals("prod"))` inside business code.
- [NEITHER] Configuration is bound to `@ConfigurationProperties` records and validated at startup. The app fails fast on a missing or malformed value — better to fail at boot than at 2 a.m. on a code path nobody exercised.

10. Maintainability & complexity

- [LINTER] **Methods under roughly 40 lines, classes under 300, nesting no deeper than three. Cyclomatic complexity gate at 10.** Check: Checkstyle, SpotBugs.
- [LINTER] **Constructor injection, fields `final`.** A constructor with nine arguments is the class telling you it does too much — that is a feature of this rule, not a nuisance. Check: ArchUnit — no `@Autowired` on fields.
- [NEITHER] **`@Transactional` belongs on the service method.** Never on a controller, never on a private or self-invoked method — the proxy does not apply and it fails silently.
- [NEITHER] **Associations `LAZY` by default.** Fix N+1 with `@EntityGraph` or a join fetch, never by switching to `EAGER`.
- [LINTER] **Every list endpoint is paginated.** No unbounded `findAll()` on a table that grows. Use a projection when you need three columns out of thirty.
- [NEITHER] **Extract on the third copy, not the second.** Premature abstraction costs more than a little duplication.
- [NEITHER] **Public endpoints carry OpenAPI annotations. Comments explain *why*, never *what*.** Each README covers setup, required environment variables, and how to run locally against a Docker Postgres.

11. What we do NOT do

- [LINTER] **Field `@Autowired`.** Untestable without Spring, and it hides how many dependencies a class has grown.
- [LINTER] **Lombok `@Data` on an entity.** The generated `equals` / `hashCode` / `toString` trigger lazy loads and break the entity inside a `Set`. Use `@Getter` / `@Setter`.
- [LINTER] **`ddl-auto: update` outside local.** The fastest known way to lose a production column.
- [LINTER] **Entities in request or response bodies.** No exceptions, including “it is just an internal endpoint”.
- [NEITHER] **`Map<String, Object>` as a response type.** If it has no type, it has no contract.
- [NEITHER] **HTTP 200 with `{"success": false}`.** Status codes exist; use them.
- [NEITHER] **Verbs in URLs — `/api/getPropertyList`.** Nouns plus HTTP methods.
- [NEITHER] **`@Transactional` around an external API call.** The DB connection is held hostage for the whole remote timeout and the pool drains.
- [LINTER] **`System.out.println`, `show-sql` in production, and commented-out code.** Git already remembers.
- [NEITHER] **New `Util` / `Helper` / `Manager` classes.** They start as one static method and end as the place nobody wants to open.
- [NEITHER] **`double` or `float` for money.** See section 12.
- [NEITHER] **Writing log files from the application.** Logs go to stdout; the platform collects them.

12. Money and numeric values

[NEITHER] **`double` and `float` never touch a monetary or token amount.** Not in an entity, not in a DTO, not in a calculation, not in a column.

`0.1 + 0.2` is `0.30000000000000004`. On a financial platform that produces defects found by clients, not by us.

Kind	Type	Why
Fiat currency	<code>Long</code> minor units, serialised as <code>String</code>	Exact, and matches what Node and Python send
Rates and percentages	integer basis points (<code>interestBps = 250 = 2.5%</code>)	No decimal to round
Intermediate division	<code>BigDecimal</code> with an explicit <code>MathContext</code>	Rounding is a decision, not an accident
Database column	<code>NUMERIC</code> / <code>BIGINT</code>	Never <code>FLOAT</code> or <code>DOUBLE PRECISION</code>

```
// common/money/Money.java
public final class Money {

    private Money() {}
```

```

/** Minor units. 1050L == ₹10.50. Integer arithmetic throughout. */
public static long applyBps(long amountMinor, int bps) {
    if (bps < 0) throw new IllegalArgumentException("bps must be non-negative");
    // multiplyExact throws on overflow rather than wrapping silently.
    return Math.multiplyExact(amountMinor, (long) bps) / 10_000L;
}

/** The remainder goes to the first instalment. Decided once, here. */
public static long[] splitEvenly(long amountMinor, int parts) {
    long base = amountMinor / parts;
    long remainder = amountMinor % parts;
    var out = new long[parts];
    for (int i = 0; i < parts; i++) out[i] = base + (i < remainder ? 1 : 0);
    return out;
}

/** JSON numbers lose precision on large values. Money crosses the wire as a string. */
public static String toWire(long amountMinor) {
    return Long.toString(amountMinor);
}
}

```

- [NEITHER] **Amounts cross the wire as strings.** The frontends treat money as strings; a JSON number is not safe above 2^{53} .
- [NEITHER] **Use `Math.multiplyExact` and `Math.addExact` on amounts.** Silent overflow on a `long` is a worse failure than an exception.
- [NEITHER] **Rounding is decided once, written down, and applied in one place** — which direction, at which step, and who absorbs the remainder. Two developers rounding independently is how a ledger stops balancing.
- [NEITHER] `BigDecimal` is used only where division demands it, always with an explicit scale and `RoundingMode`. **Never `new BigDecimal(double)`** — it captures the float's error. Use `BigDecimal.valueOf` or the `String` constructor.
- [NEITHER] The backend is authoritative for every total, fee and interest figure.

13. Data access & transactions

13.1 Connection pool and session lifetime

- [NEITHER] **HikariCP pool size is configured deliberately and documented.** The default is rarely right for the instance size. Pool exhaustion is an alerting event (4.4).
- [NEITHER] `spring.jpa.open-in-view` is `false`. Left at its default, the persistence session stays open through view rendering, which hides lazy-loading problems in development and produces N+1 under load in production.
- [NEITHER] Long-running work — a report, a batch job — does not hold a request-scoped connection. It runs on its own pool or in a separate job.

```

spring:
  jpa:
    open-in-view: false
    hibernate:
      ddl-auto: validate          # never update, above local
    datasource:
      hikari:
        maximum-pool-size: 10     # sized for the instance, not left at default
        connection-timeout: 3000
        leak-detection-threshold: 20000

```

13.2 Transactions

- [NEITHER] **Multiple writes that must succeed or fail together go in one transaction.**

If a failure partway through would leave related records inconsistent — money moved but status not updated, a record created but a linked one missing — the writes go in a transaction. *A partial write is a data-integrity bug that surfaces days later as a support ticket nobody can explain.*

- [NEITHER] `@Transactional` on the service method; `readOnly = true` on reads.
- [NEITHER] **Never on a private or self-invoked method.** The proxy does not apply and it fails silently — the code looks transactional and is not.
- [NEITHER] **Never around an external API call.**
- [NEITHER] A migration ships with a rollback plan and is safe to run on a populated table.

13.3 Idempotency

[NEITHER] Any operation that can be retried carries an idempotency key, and a repeat returns the original result rather than performing the work again.

Clients retry. Load balancers retry. Payment providers redeliver. A retried transfer is not a harmless retry.

- **[NEITHER]** The key and the result are written **in the same transaction as the work**. Written afterwards, a crash in between leaves the work done and the key missing.
- **[NEITHER]** Webhook handlers key on the provider's event id and tolerate duplicates.

14. Async & scheduled work

- **[NEITHER]** A failed `@Scheduled` or batch job raises an alert (4.4). A job that fails silently compounds daily and is found by a client.
- **[NEITHER]** `@Scheduled` methods are idempotent, and in a multi-instance deployment they use a lock (`ShedLock` or equivalent). Two instances running the same job at the same time is the default behaviour, not an edge case.
- **[NEITHER]** `@Async` methods return `CompletableFuture` and their exceptions are handled. An `@Async void` method that throws loses the exception entirely — nothing logs it, nothing alerts.
- **[NEITHER]** Every outbound HTTP call sets a **connect and read timeout**. A call without one can hang until the thread pool is exhausted.
- **[NEITHER]** `@Async` and `@Transactional` on the same method do not behave as written. The transaction belongs on the method the async task calls, not on the async entry point.
- **[NEITHER]** Long jobs log progress and have a timeout. A job with no timeout and no output is indistinguishable from a hung one.

Appendix A — ArchUnit tests

These enforce section 2 and parts of 3, 10 and 11 as part of the normal test run. Add the rules incrementally: a rule that fails on existing code is marked `@ArchIgnore` with a dated comment until that code is cleaned up.

```
@AnalyzeClasses(packages = "com.company.project", importOptions = ImportOption.DoNotIncludeTests.class)
class ArchitectureTest {

    @ArchTest
    static final ArchRule layering = layeredArchitecture().consideringAllDependencies()
        .layer("Controller").definedBy("..controller..")
        .layer("Service").definedBy("..service..")
        .layer("Repository").definedBy("..repository..")
        .whereLayer("Controller").mayNotBeAccessedByAnyLayer()
        .whereLayer("Service").mayOnlyBeAccessedByLayers("Controller")
        .whereLayer("Repository").mayOnlyBeAccessedByLayers("Service");

    @ArchTest
    static final ArchRule controllersDoNotTouchRepositories =
        noClasses().that().resideInAPackage("..controller..")
            .should().dependOnClassesThat().resideInAPackage("..repository..")
            .because("a controller must never inject a repository (standard 2)");

    @ArchTest
    static final ArchRule entitiesDoNotLeave =
        noMethods().that().areDeclaredInClassesThat().resideInAPackage("..controller..")
            .should().haveRawReturnType(resideInAPackage("..entity.."))
            .because("returning an entity makes every column public API (standard 2)");

    @ArchTest
    static final ArchRule noFieldInjection =
        noFields().should().beAnnotatedWith(Autowired.class)
            .because("constructor injection only (standard 10)");

    @ArchTest
    static final ArchRule noTryCatchInControllers =
        noClasses().that().resideInAPackage("..controller..")
            .should().callMethodWhere(target(nameMatching("printStackTrace")))
            .because("errors go through the advice (standard 3)");

    @ArchTest
    static final ArchRule transactionalOnServicesOnly =
        methods().that().areAnnotatedWith(Transactional.class)
```

```

        .should().beDeclaredInClassesThat().resideInAPackage("..service..")
        .andShould().bePublic()
        .because("a non-public or controller @Transactional fails silently (standard 13.2)");

@ArchTest
static final ArchRule noFloatMoney =
    noFields().that().haveNameMatching(".*([Aa]mount|[Pp]rice|[Bb]alance|[Ff]ee|[Tt]otal).")
        .should().haveRawType(double.class).orShould().haveRawType(float.class)
        .because("money is long minor units (standard 12)");

@ArchTest
static final ArchRule noUtilClasses =
    noClasses().should().haveSimpleNameEndingWith("Util")
        .orShould().haveSimpleNameEndingWith("Helper")
        .orShould().haveSimpleNameEndingWith("Manager")
        .because("they become the place nobody wants to open (standard 11)");
}

```

The money rule is a name-based heuristic and will miss amounts named otherwise. It is a prompt to look, not a verdict — the real check is review item 7.

Appendix B — JaCoCo coverage gate

```

<plugin>
  <groupId>org.jacoco</groupId>
  <artifactId>jacoco-maven-plugin</artifactId>
  <executions>
    <execution>
      <id>check</id>
      <goals><goal>check</goal></goals>
      <configuration>
        <rules>
          <rule>
            <element>PACKAGE</element>
            <includes>
              <include>com.company.project.*.service.*</include>
              <include>com.company.project.*.domain.*</include>
            </includes>
            <limits>
              <limit>
                <counter>LINE</counter>
                <value>COVEREDRATIO</value>
                <minimum>0.70</minimum>
              </limit>
            </limits>
          </rule>
        </rules>
      </configuration>
    </execution>
  </executions>
</plugin>

```

DTOs, entities and config are excluded deliberately. Coverage on getters is a vanity number.